ITS ICT Linux	**DOCENTE:** _____
	UNITÀ FORMATIVA: Strumenti Linux per la gestione dei file
	ARGOMENTO: Ricerca nei file: grep, sed, awk

Introduzione

Linux mette a disposizione tre strumenti fondamentali per **cercare e manipolare il testo** nei file:

- grep — cerca righe che contengono un pattern
- sed — trasforma il testo (cerca e sostituisce)
- awk — elabora testo strutturato a colonne

I file di esempio utilizzati in questa lezione si trovano nella cartella `~/esercizi/`.

1. grep — Cercare nei file

Il comando **`grep` (Global Regular Expression Print)** cerca le righe di un file che corrispondono a un pattern e le stampa sullo standard output. **Non modifica mai i file.**

Sintassi

```
$ grep [opzioni] "pattern" file
```

Opzioni principali

Opzione	Significato
`-i`	Ricerca case-insensitive (maiuscole = minuscole)
`-r`	Ricerca ricorsiva in tutte le sottodirectory
`-n`	Mostra il numero di riga prima di ogni risultato
`-v`	Inverte la ricerca: mostra le righe che NON corrispondono
`-l`	Mostra solo i nomi dei file con almeno una corrispondenza
`-c`	Conta il numero di righe corrispondenti
`-E`	Abilita le espressioni regolari estese (ERE)
`-w`	Cerca solo parole intere (word match)

Esempi pratici

Ricerca semplice

```
$ grep "ERROR" sistema.log
```

Cerca tutte le righe che contengono la parola `ERROR` nel file `sistema.log`.

Ricerca case-insensitive con numero di riga

```
$ grep -n -i "error" sistema.log
```

Trova sia `ERROR` sia `error` sia `Error`, mostrando il numero di riga per ogni risultato.

Ricerca invertita

```
$ grep -v "^#" webserver.conf
```
Mostra tutte le righe del file di configurazione che **non** iniziano con `#` (cioè esclude i commenti).

Ricerca ricorsiva

```
$ grep -rl "password" /etc/
```
Cerca la parola `password` in tutti i file dentro `/etc/` e mostra solo i nomi dei file trovati.

Uso di espressioni regolari

```
$ grep -E "ERROR|WARNING" sistema.log
```
Cerca le righe che contengono `ERROR` oppure `WARNING` (operatore `|` = OR).

> **NOTA:** grep non modifica mai i file. Serve solo per trovare e visualizzare righe. Per modificare il testo si usa sed.

Ancore nelle espressioni regolari

Ancora	Esempio	Significato
`^`	`grep "^root" utenti.txt`	Righe che iniziano con "root"
`$`	`grep "bash$" utenti.txt`	Righe che terminano con "bash"
`.`	`grep "er.ore" file.txt`	Il punto sostituisce un qualsiasi carattere

2. sed — Modificare il testo

Il comando **`sed` (Stream EDitor)** legge un file riga per riga, applica una trasformazione e stampa il risultato. **Di default non modifica il file originale**: l'output va sullo standard output. Con l'opzione `-i` la modifica avviene direttamente sul file.

Sintassi

```
$ sed [opzioni] 'comando' file
```

Il comando di sostituzione: s///

Il comando più usato di sed è **`s`** (substitute):

```
$ sed 's/testo_vecchio/testo_nuovo/flag' file
```

Flag	Significato
`(nessuno)`	Sostituisce solo la prima occorrenza per ogni riga
`g`	Sostituisce tutte le occorrenze nella riga (global)
`i`	Sostituzione case-insensitive
`2`	Sostituisce solo la seconda occorrenza

Esempi pratici

Sostituzione semplice (output su terminale)

```
$ sed 's/WARNING/ATTENZIONE/' sistema.log
```

Mostra il file con `WARNING` sostituito da `ATTENZIONE` nella prima occorrenza di ogni riga. Il file originale non viene modificato.

Sostituzione con modifica in-place

```
$ sed -i 's/porta = 80/porta = 8080/' webserver.conf
```

Modifica direttamente il file `webserver.conf`, cambiando il valore della porta.

> **ATTENZIONE:** L'opzione -i modifica il file in modo irreversibile. Prima di usarla, verificare il risultato senza -i per accertarsi che la sostituzione sia corretta.

Copia di sicurezza automatica con -i

```
$ sed -i.bak 's/WARNING/ATTENZIONE/g' sistema.log
```

Crea automaticamente una copia di backup `sistema.log.bak` prima di modificare il file.

Eliminare righe che corrispondono a un pattern

```
$ sed '/^#/d' webserver.conf
```

Il comando `d` (delete) elimina le righe che corrispondono al pattern. In questo caso elimina tutti i commenti dall'output.

Visualizzare solo un intervallo di righe

```
$ sed -n '1,5p' sistema.log
```

Con `-n` (no print) e il comando `p` mostra solo le righe dalla 1 alla 5.

Workflow consigliato: grep + sed

Passo 1 — verificare cosa verrà modificato con grep:

```
$ grep "livello = WARNING" webserver.conf
```

Passo 2 — eseguire la sostituzione con sed (senza -i prima):

```
$ sed 's/livello = WARNING/livello = ERROR/' webserver.conf
```

Passo 3 — se l'output è corretto, applicare con -i:

```
$ sed -i 's/livello = WARNING/livello = ERROR/' webserver.conf
```

3. awk — Elaborare testo strutturato

Il comando **`awk`** è uno strumento di elaborazione del testo ottimizzato per file con **dati a colonne** (log, CSV, output di comandi). Divide ogni riga in campi e permette di selezionare, filtrare e calcolare.

Sintassi

```
$ awk [opzioni] 'pattern { azione }' file
```

Variabili predefinite

Variabile	Significato
`$0`	L'intera riga corrente
`$1, $2, ...`	Il primo, secondo, ... campo della riga
`NF`	Numero totale di campi nella riga corrente

`NR`	Numero di riga corrente (a partire da 1)
`FS`	Field Separator: separatore di campo (default: spazio). Si imposta con `-F`

Esempi pratici

Stampare una colonna specifica

```
$ awk '{print $1}' sistema.log
```

Stampa solo la prima colonna (la data) di ogni riga del log.

Usare un separatore personalizzato

```
$ awk -F: '{print $1, $3}' utenti.txt
```

Usa `:` come separatore di campo e stampa nome utente (campo 1) e UID (campo 3).

Filtrare per valore di un campo

```
$ awk -F: '$3 > 1000 {print $1, $3, $6}' utenti.txt
```

Mostra solo gli utenti con UID maggiore di 1000 (utenti normali), con nome, UID e home directory.

Lavorare con file CSV

```
$ awk -F, '$3 == "fotocamere" {print $2, $6}' vendite.csv
```

Mostra nome prodotto e totale solo per le vendite della categoria `fotocamere`.

Calcolare la somma di una colonna

```
$ awk -F, 'NR>1 {sum += $6} END {print "Totale vendite:", sum}' vendite.csv
```

Somma tutti i valori del campo `totale` (colonna 6) saltando la riga di intestazione (`NR>1`), poi stampa il risultato alla fine.

Contare le occorrenze per categoria

```
$ awk -F, 'NR>1 {contatore[$3]++} END {for (cat in contatore) print cat,
contatore[cat]}' vendite.csv
```

Conta quante vendite ci sono per ogni categoria usando un array associativo.

4. Combinare gli strumenti in pipeline

La vera potenza di grep, sed e awk emerge quando vengono **combinati in pipeline** con `|`.

Trovare i file modificati di recente che contengono un pattern

```
$ find /etc -name "*.conf" -mtime -7 | xargs grep -l "porta"
```

Estrarre gli IP che hanno fallito il login

```
$ grep "Failed password" sistema.log | awk '{print $11}' | sort | uniq -c | sort -rn
```

Estrae le righe con login falliti, prende l'IP (campo 11), conta le occorrenze e le ordina per frequenza decrescente.

Mostrare gli utenti reali con la loro shell

```
$ grep -v "nologin\|false" utenti.txt | awk -F: '{print $1, "→", $7}'
```

Esclude gli utenti di sistema e mostra solo quelli con una shell interattiva reale.

5. Riepilogo comparativo

Comando	Modifica file?	Scopo principale	Quando usarlo
`grep`	**No**	Cercare pattern nel testo	Trovare righe che contengono una stringa o pattern
`sed`	**Sì con -i**	Modificare il testo (sostituire, eliminare)	Sostituire testo, eliminare righe, trasformare stream
`awk`	**No**	Elaborare testo a colonne	Analizzare CSV, log, calcoli su colonne

6. Esercizi

File disponibili nella cartella `~/esercizi/`**:**

File	Contenuto
`sistema.log`	Log di sistema con eventi, errori e accessi SSH
`utenti.txt`	File in formato /etc/passwd con utenti di sistema e reali
`webserver.conf`	File di configurazione di un server web con commenti
`vendite.csv`	Dati di vendita in formato CSV con intestazione

Esercizi grep

Es. 1 — Mostrare tutte le righe di `sistema.log` che contengono la parola `ERROR`.

Es. 2 — Mostrare le righe di `sistema.log` che contengono `ERROR` oppure `WARNING` (usare `-E`).

Es. 3 — Contare quante volte appare `Failed password` nel file di log.

Es. 4 — In `utenti.txt` mostrare solo gli utenti la cui shell è `/bin/bash`.

Es. 5 — In `webserver.conf` mostrare tutte le righe non vuote e non commentate (escludere righe che iniziano con `#` o sono vuote).

Esercizi sed

Es. 6 — Nel file `webserver.conf`, sostituire `WARNING` con `ERROR` solo a video (senza modificare il file).

Es. 7 — Nel file `webserver.conf`, eliminare tutte le righe di commento (che iniziano con `#`) dall'output.

Es. 8 — Nel file `sistema.log`, sostituire `server01` con `webserver.lan` e salvare il risultato in un nuovo file `sistema_mod.log`.

Esercizi awk

Es. 9 — Da `utenti.txt`, estrarre e stampare solo il nome utente (campo 1) e la home directory (campo 6) usando `:` come separatore.

Es. 10 — Da `vendite.csv`, mostrare solo le righe della categoria `obiettivi` con prodotto e prezzo unitario.

Es. 11 — Da `vendite.csv`, calcolare il totale delle vendite per ogni venditore.

Es. 12 (avanzato) — Da `sistema.log`, estrarre tutti gli indirizzi IP che compaiono nel log e contare quante volte appare ciascuno.